5.Predict the price of the Uber ride from a given pickup point to the agreed drop-off location. Perform following tasks: 1. Pre-process the dataset. 2. Identify outliers. 3. Check the correlation. 4. Implement linear regression and ridge, Lasso regression models. 5. Evaluate the models and compare their respective scores like R2, RMSE, etc.

```
import pandas as pd
df=pd.read_csv("/content/uber.csv")
df
```

	Unnamed: 0	key	fare_amount	pickup_datetime	pickup_longitude	pickup_latitude	dropoff_longitude	dro
0	24238194	2015-05-07 19:52:06.0000003	7.5	2015-05-07 19:52:06 UTC	-73.999817	40.738354	-73.999512	
1	27835199	2009-07-17 20:04:56.0000002	7.7	2009-07-17 20:04:56 UTC	-73.994355	40.728225	-73.994710	
2	44984355	2009-08-24 21:45:00.00000061	12.9	2009-08-24 21:45:00 UTC	-74.005043	40.740770	-73.962565	
3	25894730	2009-06-26 08:22:21.0000001	5.3	2009-06-26 08:22:21 UTC	-73.976124	40.790844	-73.965316	
4	17610152	2014-08-28 17:47:00.000000188	16.0	2014-08-28 17:47:00 UTC	-73.925023	40.744085	-73.973082	
...	...	...	...	...	...	...	...	
199995	42598914	2012-10-28 10:49:00.00000053	3.0	2012-10-28 10:49:00 UTC	-73.987042	40.739367	-73.986525	
199996	16382965	2014-03-14 01:09:00.0000008	7.5	2014-03-14 01:09:00 UTC	-73.984722	40.736837	-74.006672	
199997	27804658	2009-06-29 00:42:00.00000078	30.9	2009-06-29 00:42:00 UTC	-73.986017	40.756487	-73.858957	
199998	20259894	2015-05-20 14:56:25.0000004	14.5	2015-05-20 14:56:25 UTC	-73.997124	40.725452	-73.983215	
199999	11951496	2010-05-15 04:08:00.00000076	14.1	2010-05-15 04:08:00 UTC	-73.984395	40.720077	-73.985508	

200000 rows × 9 columns

```
import matplotlib.pyplot as plt
import seaborn as sns
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression, Ridge, Lasso
from sklearn.metrics import r2_score, mean_squared_error
```

```
df.head()
df.tail()
```

	Unnamed: 0	key	fare_amount	pickup_datetime	pickup_longitude	pickup_latitude	dropoff_longitude	drop
199995	42598914	2012-10-28 10:49:00.00000053	3.0	2012-10-28 10:49:00 UTC	-73.987042	40.739367	-73.986525	
199996	16382965	2014-03-14 01:09:00.0000008	7.5	2014-03-14 01:09:00 UTC	-73.984722	40.736837	-74.006672	
199997	27804658	2009-06-29 00:42:00.00000078	30.9	2009-06-29 00:42:00 UTC	-73.986017	40.756487	-73.858957	
199998	20259894	2015-05-20 14:56:25.0000004	14.5	2015-05-20 14:56:25 UTC	-73.997124	40.725452	-73.983215	
199999	11951496	2010-05-15 04:08:00.00000076	14.1	2010-05-15 04:08:00 UTC	-73.984395	40.720077	-73.985508	

```
df.columns
```

```
Index(['Unnamed: 0', 'key', 'fare_amount', 'pickup_datetime',
       'pickup_longitude', 'pickup_latitude', 'dropoff_longitude',
       'dropoff_latitude', 'passenger_count'],
      dtype='object')
```

```
df.describe()
```

	Unnamed: 0	fare_amount	pickup_longitude	pickup_latitude	dropoff_longitude	dropoff_latitude	passenger_count
count	2.000000e+05	200000.000000	200000.000000	200000.000000	199999.000000	199999.000000	200000.000000
mean	2.771250e+07	11.359955	-72.527638	39.935885	-72.525292	39.923890	1.684535
std	1.601382e+07	9.901776	11.437787	7.720539	13.117408	6.794829	1.385997
min	1.000000e+00	-52.000000	-1340.648410	-74.015515	-3356.666300	-881.985513	0.000000
25%	1.382535e+07	6.000000	-73.992065	40.734796	-73.991407	40.733823	1.000000
50%	2.774550e+07	8.500000	-73.981823	40.752592	-73.980093	40.753042	1.000000
75%	4.155530e+07	12.500000	-73.967154	40.767158	-73.963658	40.768001	2.000000
max	5.542357e+07	499.000000	57.418457	1644.421482	1153.572603	872.697628	208.000000

Remove Missing Values

```
df.isnull().sum()
```

	0
Unnamed: 0	0
key	0
fare_amount	0
pickup_datetime	0
pickup_longitude	0
pickup_latitude	0
dropoff_longitude	1
dropoff_latitude	1
passenger_count	0

dtype: int64

```
df = df.dropna()
df = df[df['fare_amount'] > 0]
```

```
(199977, 9)
```

Data Cleaning

Remove Invalid Fare Values

```
df=df[df['passenger_count']<=0]
```

Feature Engineering (Distance Calculation)

```
df=df[
    (df['pickup_latitude'].between(-90,90))&
    (df['pickup_longitude'].between(-180,180))&
    (df['dropoff_latitude'].between(-90,90))&
    (df['dropoff_longitude'].between(-180,180))
]
```

```
df['pickup_datetime']=pd.to_datetime(df['pickup_datetime'])
```

```
df['hour']=df['pickup_datetime'].dt.hour
df['day']=df['pickup_datetime'].dt.day
df['month']=df['pickup_datetime'].dt.month
```

```
df.drop(['pickup_datetime', 'key'], axis=1, inplace=True)
```

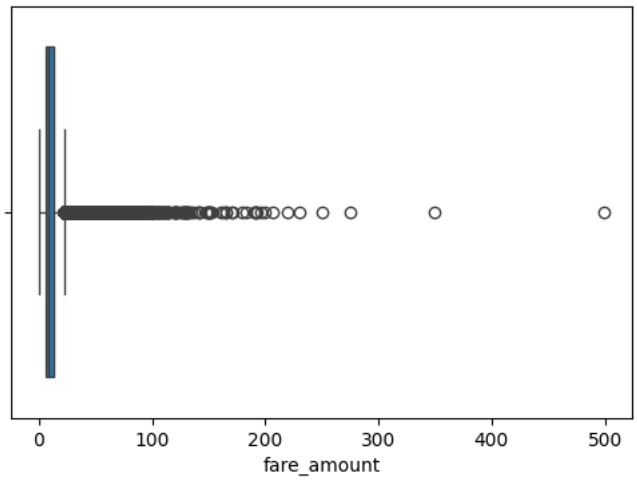
Havrsine formula

```
def haversine(lat1, lon1, lat2, lon2):
    lat1, lon1, lat2, lon2 = map(np.radians, [lat1, lon1, lat2, lon2])
    dlat = lat2 - lat1
    dlon = lon2 - lon1
    a = np.sin(dlat/2)**2 + np.cos(lat1)*np.cos(lat2)*np.sin(dlon/2)**2
    c = 2*np.arcsin(np.sqrt(a))
    r = 6371
    return c * r
```

```
df['distance'] = haversine(
    df['pickup_latitude'],
    df['pickup_longitude'],
    df['dropoff_latitude'],
    df['dropoff_longitude']
)
```

Using Boxplot

```
plt.figure(figsize=(6,4))
sns.boxplot(x=df['fare_amount'])
plt.show()
```



Remove Outliers Using IQR Method

```
Q1 = df['fare_amount'].quantile(0.25)
Q3 = df['fare_amount'].quantile(0.75)
IQR = Q3 - Q1

df = df[(df['fare_amount'] >= Q1 - 1.5*IQR) &
        (df['fare_amount'] <= Q3 + 1.5*IQR)]
```

Check Correlation

```
df.corr(numeric_only=True)
```

	Unnamed: 0	fare_amount	pickup_longitude	pickup_latitude	dropoff_longitude	dropoff_latitude	passenger_c
Unnamed: 0	1.000000	-0.000158	0.000455	-0.001166	0.000838	-0.000491	0.00
fare_amount	-0.000158	1.000000	0.000425	0.002573	-0.002299	0.000466	0.01
pickup_longitude	0.000455	0.000425	1.000000	-0.812017	0.835992	-0.850629	-0.00
pickup_latitude	-0.001166	0.002573	-0.812017	1.000000	-0.767133	0.684576	-0.00
dropoff_longitude	0.000838	-0.002299	0.835992	-0.767133	1.000000	-0.913919	0.00
dropoff_latitude	-0.000491	0.000466	-0.850629	0.684576	-0.913919	1.000000	-0.00
passenger_count	0.002461	0.012618	-0.000836	-0.001537	0.000124	-0.000768	1.00
distance	0.000881	0.002766	0.098303	-0.049154	0.038700	-0.064244	-0.00

```
df.corr(method='pearson')
```

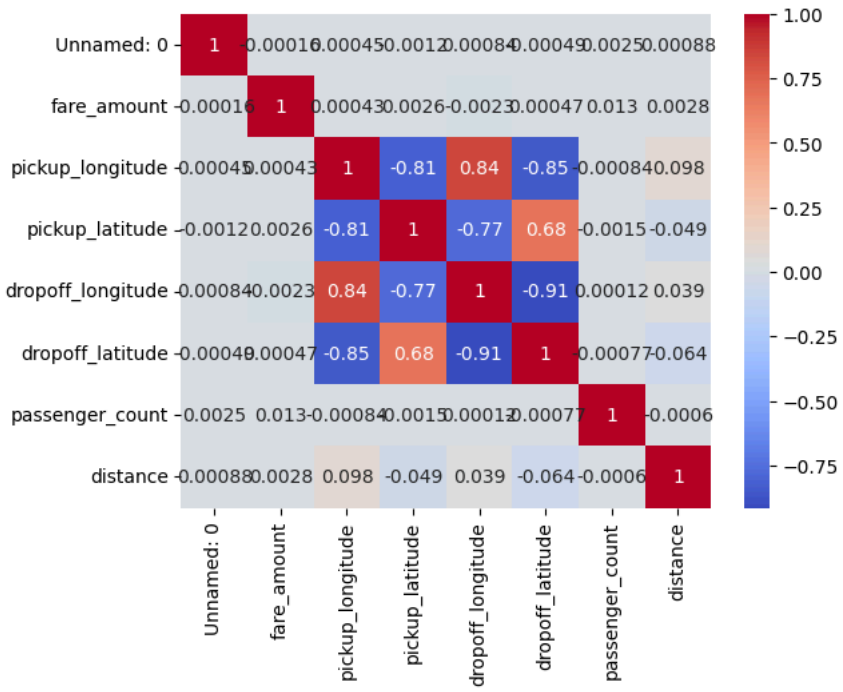
	Unnamed: 0	fare_amount	pickup_longitude	pickup_latitude	dropoff_longitude	dropoff_latitude	passenger_c
Unnamed: 0	NaN	NaN	NaN	NaN	NaN	NaN	
fare_amount	NaN	NaN	NaN	NaN	NaN	NaN	
pickup_longitude	NaN	NaN	NaN	NaN	NaN	NaN	
pickup_latitude	NaN	NaN	NaN	NaN	NaN	NaN	
dropoff_longitude	NaN	NaN	NaN	NaN	NaN	NaN	
dropoff_latitude	NaN	NaN	NaN	NaN	NaN	NaN	
passenger_count	NaN	NaN	NaN	NaN	NaN	NaN	
hour	NaN	NaN	NaN	NaN	NaN	NaN	
day	NaN	NaN	NaN	NaN	NaN	NaN	
month	NaN	NaN	NaN	NaN	NaN	NaN	
distance	NaN	NaN	NaN	NaN	NaN	NaN	

```
df.corr(method='kendall')
```

	Unnamed: 0	fare_amount	pickup_longitude	pickup_latitude	dropoff_longitude	dropoff_latitude	passenger_c
Unnamed: 0	NaN	NaN	NaN	NaN	NaN	NaN	
fare_amount	NaN	NaN	NaN	NaN	NaN	NaN	
pickup_longitude	NaN	NaN	NaN	NaN	NaN	NaN	
pickup_latitude	NaN	NaN	NaN	NaN	NaN	NaN	
dropoff_longitude	NaN	NaN	NaN	NaN	NaN	NaN	
dropoff_latitude	NaN	NaN	NaN	NaN	NaN	NaN	
passenger_count	NaN	NaN	NaN	NaN	NaN	NaN	
hour	NaN	NaN	NaN	NaN	NaN	NaN	
day	NaN	NaN	NaN	NaN	NaN	NaN	
month	NaN	NaN	NaN	NaN	NaN	NaN	
distance	NaN	NaN	NaN	NaN	NaN	NaN	

```
import seaborn as sns
import matplotlib.pyplot as plt

sns.heatmap(df.corr(numeric_only=True), annot=True, cmap='coolwarm')
plt.show()
```



Select Features and Target

```
X = df[['distance']]
y = df['fare_amount']
```

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

```
from sklearn.preprocessing import StandardScaler
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

Linear Regression

```
from sklearn.linear_model import LinearRegression
lr = LinearRegression()
lr.fit(X_train, y_train)
y_pred_lr = lr.predict(X_test)
```

Ridge Regression

```
from sklearn.linear_model import Ridge
ridge = Ridge(alpha=1.0)
ridge.fit(X_train_scaled, y_train)
y_pred_ridge = ridge.predict(X_test_scaled)
```

Lasso Regression

```
from sklearn.linear_model import Lasso
lasso = Lasso(alpha=0.1)
lasso.fit(X_train_scaled, y_train)
y_pred_lasso = lasso.predict(X_test_scaled)
```

Model Evaluation r2 score

```
from sklearn.metrics import r2_score
print("Linear Regression R2:", r2_score(y_test, y_pred_lr))
print("Ridge Regression R2:", r2_score(y_test, y_pred_ridge))
print("Lasso Regression R2:", r2_score(y_test, y_pred_lasso))
```

```
Linear Regression R2: -0.00014032982471112732
Ridge Regression R2: -0.00014032976161937327
Lasso Regression R2: -0.00013119214248935585
```

RMSE

```
from sklearn.metrics import mean_squared_error
print("Linear RMSE:", np.sqrt(mean_squared_error(y_test, y_pred_lr)))
print("Ridge RMSE:", np.sqrt(mean_squared_error(y_test, y_pred_ridge)))
print("Lasso RMSE:", np.sqrt(mean_squared_error(y_test, y_pred_lasso)))
```

```
Linear RMSE: 4.1755735386197355
Ridge RMSE: 4.175573538488031
Lasso RMSE: 4.175554463720893
```

```
results = pd.DataFrame({
    'Model': ['Linear Regression', 'Ridge Regression', 'Lasso Regression'],
    'R2 Score': [
        r2_score(y_test, y_pred_lr),
        r2_score(y_test, y_pred_ridge),
        r2_score(y_test, y_pred_lasso)
    ],
    'RMSE': [
        np.sqrt(mean_squared_error(y_test, y_pred_lr)),
        np.sqrt(mean_squared_error(y_test, y_pred_ridge)),
        np.sqrt(mean_squared_error(y_test, y_pred_lasso))
    ]
})
results
```

1 to 3 of 3 entries Filter

index	Model	R2 Score	RMSE
0	Linear Regression	-0.00014032982471112732	4.1755735386197355
1	Ridge Regression	-0.00014032976161937327	4.175573538488031
2	Lasso Regression	-0.00013119214248935585	4.175554463720893

Show 25 per page

Like what you see? Visit the data table notebook to learn more about interactive tables.

Next steps: Generate code with results New interactive sheet